1 Order dependence in `pangraph build`

Branch `debug/order-dependence`*. Investigation of the report in* `tmp/pangraph_order_bug/`*.*

1.1 Summary

`pangraph build` produces a materially different graph depending only on the order the input FASTA files are listed on the command line — 452 to 509 blocks for the same three genomes, a ~ 12% spread. The reported hypothesis was that graph merging follows argument order rather than the guide tree. That is *not* what happens: the guide tree is honoured exactly.

The real cause is that block identifiers are assigned from the input position, those identifiers are handed to `minimap2` as sequence names, and `minimap2` is invoked with `-X`, which keeps only one direction of each pairwise alignment — chosen by *string comparison of those names*. Argument order therefore decides, for every pair of blocks, which one is the query and which one is the reference. That choice is not neutral: the alignment is asymmetric, and downstream the reference is privileged when choosing the consensus that the merged block inherits.

The fix, now implemented, is to make the aligner boundary *canonical*: order and name blocks by a hash of their consensus sequence, so that everything the aligner sees is a pure function of the block content, independent of input order, tree order and identifier assignment. A second, initially underestimated defect had to be fixed alongside it - neighbor joining resolves its Q-matrix ties by matrix row order, which for three taxa is always tied, so the inferred topology itself followed the argument order. All six permutations now agree, with or without a guide tree.

Two claims in the first draft of this note did not survive measurement, and are corrected in place: the mmseqs backend did *not* share the defect, and neighbor-joining ties are not rare. The first of those led to a simpler alternative — enabling minimap2′s dual mappings — which was tested, works, and is benchmarked against the landed fix in the final section. That benchmark in turn corrected a third claim of this note's own: the apparent quality advantage of dual mappings was a confounded comparison, and disappears once only one variable is changed.

1.2 Evidence

1.2.1 The guide tree is obeyed

Running orders `ERS990151 AP025961 NZ_CP035927` and `NZ_CP035927 ERS990151 AP025961` with the same `--guide-tree` yields identical logged topology and an identical merge schedule:

```
Guide tree (newick): ((ERS990151,AP025961),NZ_CP035927);
=== Graph merging start:     clades sizes 1 + 1
=== Graph merging completed: clades sizes 1 + 1 -> 2
=== Graph merging start:     clades sizes 2 + 1
=== Graph merging completed: clades sizes 2 + 1 -> 3
```

`build_tree_from_newick` (`tree/newick.rs:70`) attaches graphs to leaves by name through a `BTreeMap`, so topology and left/right assignment are fixed entirely by the tree file. Merge scheduling is not the culprit.

1.2.2 Order dependence survives when merge order cannot vary

With two genomes and the tree `(ERS990151,AP025961)` there is exactly one possible merge. The result still depends on argument order:

argument order	blocks	consensus shared with other order	total consensus bp
`ERS990151 AP025961`	62	37 / 62	3 358 923
`AP025961 ERS990151`	62	37 / 62	3 358 784

Same block count, same depth multiset, but 25 of 62 blocks carry a different consensus sequence and the total consensus length differs by 139 bp. This rules out merge scheduling conclusively.

1.2.3 The alignment direction flips

A probe calling `align_with_minimap2_lib` directly on the two genomes, varying only the `BlockId` assignment, gives:

ids	hits	cross-genome direction (qry → ref)	matched bp
0, 1	217	`ERS990151 -> AP025961` × 147	263 139
1, 0	216	`AP025961 -> ERS990151` × 146	266 047
1, 2	217	`ERS990151 -> AP025961` × 147	263 139
2, 10	216	`AP025961 -> ERS990151` × 146	266 047

Three things follow. The cross-genome hits flip direction wholesale. The hit sets genuinely differ (147 vs 146 hits, ~ 2 900 matched bases), so this is not a relabelling. And only the *relative* order matters — `0,1` behaves identically to `1,2`, while `2,10` behaves like `1,0`, proving the comparison is lexicographic on the decimal string rather than numeric, since `"10" < "2"`.

1.3 Root cause

A four-step chain from argument position to alignment direction.

1. **Identifiers are the input position.** `Pangraph::singleton` (`pangraph/pangraph.rs:29`) sets `NodeId(fasta.index)`, `BlockId(fasta.index)` and `PathId(fasta.index)`, where `fasta.index` is the record's position in the concatenated input.
2. **Identifiers become aligner sequence names.** `align_with_minimap2_lib` (`align/minimap2_lib/align_with_minimap2_lib.rs:19`) stringifies the `BlockId` as the name passed to the index.
3. **`minimap2` picks the direction by string comparison.** The call sets `X: true` (same file, line 55), which raises `MM_F_NO_DUAL`. In the vendored source, `skip_seed` (`packages/minimap2-sys/minimap2/map.c:89`) drops every hit where `strcmp(qname, tname) > 0`; the header documents the flag as *"skip pairs where query name is lexicographically larger than target name"* (`minimap.h:11`). Each pair is aligned in one direction only.
4. **Query and reference are not interchangeable.** `assign_anchor_block` (`pangraph/reweave.rs:144`) resolves depth and ambiguity ties with `if ref_n <= qry_n`, so the reference wins ties and its consensus becomes the anchor that the merged block inherits (`MergePromise::solve_promise` returns `self.anchor_block`).

This also explains the reported *"the file listed last dominates"* pattern: the last-listed file receives the highest index, hence a lexicographically large name, hence it is usually the reference, hence usually the anchor — so its sequence dominates the merged consensus.

1.3.1 A second, independent defect

`align_with_minimap2_lib_impl` collects its results with `par_bridge()` (line 65). Rayon documents that *"the resulting iterator is not guaranteed to keep the order of the original iterator"* (`rayon-1.12.0/src/iter/par_bridge.rs:26`). `filter_matches` (`pangraph/graph_merging.rs:199`) then applies a *stable* sort by energy and greedily accepts non-overlapping matches, so equal-energy ties are resolved by an order Rayon explicitly does not guarantee. The reported control run was byte-identical, so this is not what was observed — but it is latent nondeterminism in the same code path and should be closed alongside.

1.4 Why this needs fixing

- **It changes the biology.** Block boundaries are where recombination breakpoints are called downstream. A 12% swing in block count driven by argument order propagates directly into biological conclusions.
- **It defeats the guide tree's purpose.** Users pass `--guide-tree` precisely to control merging and obtain comparable results. The current behaviour silently ignores that intent for the query/reference decision.
- **It is non-monotonic and unpredictable.** Because the comparison is lexicographic on decimal strings, behaviour changes discontinuously as block counts cross powers of ten. There is no mental model a user could form to anticipate it.
- **It is a latent-defect generator.** Opaque identifiers are steering an algorithmic decision. Any future change to identifier assignment silently changes scientific output.
- **`-X` discards real signal.** The two directions find measurably different homology (147 vs 146 hits). Currently one is chosen arbitrarily. This is a quality concern in its own right, pursued in the dual-mapping alternative section.

1.5 The fix

Attack the point where the identifier leaks into the algorithm: the aligner boundary. Order and name blocks by a hash of their consensus, so that the entire aligner input is a pure function of the block content multiset.

1.5.1 Canonical block naming

```
use std::collections::{BTreeMap, HashMap};
use std::hash::{Hash, Hasher};
use twox_hash::XxHash64;

/// Content-derived naming and ordering for the blocks handed to an aligner.
///
/// Names are fixed-width so that byte-wise (`strcmp`) comparison agrees with the
/// numeric `(consensus_hash, block_id)` order that `canonical_order` uses.
pub struct BlockNames {
  names: BTreeMap<BlockId, String>,
  ids: HashMap<String, BlockId>,
  keys: BTreeMap<BlockId, (u64, BlockId)>,
  order: Vec<BlockId>,
}

/// Order-independent key for a block: hash of its consensus, tie-broken by id.
fn consensus_key(id: BlockId, block: &PangraphBlock) -> (u64, BlockId) {
  let mut hasher = XxHash64::with_seed(0);
  block.consensus().hash(&mut hasher);
  (hasher.finish(), id)
}

impl BlockNames {
  pub fn from_blocks(blocks: &BTreeMap<BlockId, PangraphBlock>) -> Self {
    let keys: BTreeMap<BlockId, (u64, BlockId)> =
      blocks.iter().map(|(&id, b)| (id, consensus_key(id, b))).collect();

    let mut order: Vec<BlockId> = keys.keys().copied().collect();
    order.sort_by_key(|id| keys[id]);
```

```
    // 16 hex digits for the hash, 20 decimal digits for the id (usize::MAX has 20).
    // Both zero-padded, so lexicographic order == numeric order at every position.
    let names: BTreeMap<BlockId, String> = keys
      .iter()
      .map(|(&id, &(h, _))| (id, format!("{h:016x}_{:020}", id.0)))
      .collect();

    let ids = names.iter().map(|(&id, n)| (n.clone(), id)).collect();

    Self { names, ids, keys, order }
  }

  /// Blocks in canonical (content-derived) order.
  pub fn canonical_order(&self) -> impl Iterator<Item = BlockId> + '_ {
    self.order.iter().copied()
  }

  pub fn name(&self, id: BlockId) -> &str {
    &self.names[&id]
  }

  /// Recover the `BlockId` an aligner hit refers to. Fallible: an unknown name means
  /// the aligner returned something we did not feed it.
  pub fn id_of(&self, name: &str) -> Result<BlockId, Report> {
    self
      .ids
      .get(name)
      .copied()
      .ok_or_else(|| make_internal_report!("Aligner returned unknown sequence name
'{name}'"))
  }

  /// Order-independent sort key, for canonical tie-breaking downstream.
  pub fn sort_key(&self, id: BlockId) -> (u64, BlockId) {
    self.keys[&id]
  }
}
```

1.5.2 Plugging it into the aligner

```
pub fn align_with_minimap2_lib(
  blocks: &BTreeMap<BlockId, PangraphBlock>,
  names: &BlockNames,
  params: &AlignmentArgs,
) -> Result<Vec<Alignment>, Report> {
  // Canonical order, canonical names: nothing here depends on BlockId assignment.
  let (seq_names, seqs): (Vec<&str>, Vec<&str>) = names
    .canonical_order()
    .map(|id| (names.name(id), blocks[&id].consensus().as_str()))
    .unzip();

  align_with_minimap2_lib_impl(&seqs, &seq_names, names, params)
}
```

1.5.3 Deterministic parallel output ordering

Replace the unordered bridge with an indexed parallel iterator. `par_iter` over a slice is an `IndexedParallelIterator`, for which `collect` into a `Vec` preserves input order by construction:

```
let results: Vec<Minimap2Result> = seqs
  .par_iter()
  .zip(names_v.par_iter())
  .map_init(
    || Minimap2Mapper::new(&idx).unwrap(),
    |mapper, (seq, name)| {
      mapper
        .run_map(seq, name)
        .wrap_err_with(|| format!("When aligning sequence '{name}'"))
    },
  )
  .collect::<Result<Vec<_>, Report>>()?;
```

This is not merely a determinism fix: indexed splitting also gives Rayon better work division than `par_bridge`, which has to serialise pulls from the underlying sequential iterator behind a mutex.

`self_merge`'s other parallel stage, `mergers.into_par_iter()` (`pangraph/graph_merging.rs:145`), is already indexed (`Vec::into_par_iter`) and needs no change.

1.5.4 Canonical downstream tie-breaks

Two places consume the query/reference distinction and must stop privileging the reference.

Energy sorting in `filter_matches`, currently a stable sort whose ties fall through to vector order:

```
let alns = alns
  .iter()
  .map(|aln| (aln, alignment_energy2(aln, args)))
  .filter(|(_, energy)| *energy < 0.0)
  .sorted_by(|(a, ea), (b, eb)| {
    OrderedFloat(*ea)
      .cmp(&OrderedFloat(*eb))
      .then_with(|| names.sort_key(a.qry.name).cmp(&names.sort_key(b.qry.name)))
      .then_with(|| a.qry.interval.start.cmp(&b.qry.interval.start))
      .then_with(|| names.sort_key(a.reff.name).cmp(&names.sort_key(b.reff.name)))
      .then_with(|| a.reff.interval.start.cmp(&b.reff.interval.start))
  })
  .map(|(aln, _)| aln)
  .collect_vec();
```

Anchor selection in `assign_anchor_block`:

```
fn assign_anchor_block(mergers: &mut [Alignment], graph: &Pangraph, names: &BlockNames)
{
  for m in mergers.iter_mut() {
    let ref_block = &graph.blocks[&m.reff.name];
    let qry_block = &graph.blocks[&m.qry.name];

    let n_of = |b: &PangraphBlock, iv: &Interval| {
      b.consensus()[iv.to_range()].iter().filter(|c| c.0 == b'N').count()
    };

    // Deeper block wins; then fewer ambiguous bases; then a content-derived key.
    // The final arm replaces `ref_n <= qry_n`, which privileged the reference and
    // therefore leaked input order into the choice of surviving consensus.
    let anchor = ref_block
      .depth()
```

```
      .cmp(&qry_block.depth())
      .then_with(|| n_of(qry_block, &m.qry.interval).cmp(&n_of(ref_block,
&m.reff.interval)))
      .then_with(|| names.sort_key(m.qry.name).cmp(&names.sort_key(m.reff.name)));

    m.anchor_block = Some(match anchor {
      Ordering::Less => AnchorBlock::Qry,
      _ => AnchorBlock::Ref,
    });
  }
}
```

1.5.5 Why this is stronger than ordering identifiers by the guide tree

An obvious alternative is to assign singleton identifiers in tree-leaf order rather than FASTA order when `--guide-tree` is given. It does achieve argument-order independence, but it is strictly weaker:

- It only covers the `--guide-tree` path, leaving the default invocation broken. The neighbor-joining path has its own, independent order dependence, addressed separately below.
- It trades one arbitrary convention for another. `((A,B),C)` and `((B,A),C)` are the same topology but would still give different graphs.
- It has an implementation trap: `PathId.0` is used as a direct index into the FASTA vector (`reconstruct/reconstruct_run.rs:62` together with `build/build_run.rs:44`), so renumbering without permuting the records silently verifies against the wrong sequences.

By contrast, canonicalising the aligner boundary removes the dependence on the first two at once. Because `graph_join` is symmetric (`map_merge` over `BTreeMaps`) and `merge_graphs` uses left/right only for debug logging, sibling order in the tree also stops mattering once the aligner input is canonical — confirmed by measurement. The neighbor-joining path needs its own fix, which is the one place where this approach is not sufficient on its own.

1.6 Problems this could introduce

1.6.1 Hash collisions between identical blocks

This is the sharpest hazard, and naming purely by content hash would be a regression.

Two distinct blocks can legitimately carry an identical consensus — repeated elements, or duplicated regions in the same genome that have not yet merged. If both were given the same name, `skip_seed` (`map.c:81`) would take the `MM_F_NO_DIAG` branch:

```
cmp = strcmp(qname, s->name);
if ((flag&MM_F_NO_DIAG) && cmp == 0 && (int)s->len == qlen) {
    if ((uint32_t)r>>1 == (q->q_pos>>1)) return 1; // avoid the diagonal anchors
    ...
}
```

For two identical sequences the true alignment *is* the diagonal, so every anchor supporting it would be discarded and the pair would never merge — precisely the pair we most want merged. `find_matches` would compound this: it filters `m.qry.name != m.reff.name` to drop self-alignments, which with colliding names would also drop the genuine cross-block hit.

The `{hash}_{id}` suffix resolves this. Distinct blocks always have distinct names, so `cmp == 0` occurs only for a true self-comparison, and `NO_DIAG` retains exactly its intended meaning.

The residual is bounded and benign. When two blocks share a consensus, the direction falls back to `BlockId`, which is still input-order dependent. But the two sequences are identical, so the alignment

is symmetric and the anchor choice affects only which *identifier* survives, not the consensus. Downstream, identifiers no longer influence alignment, so the effect is confined to labels in the output JSON. This should be stated in the docs rather than papered over: invariance is guaranteed when consensus sequences are distinct.

A true 64-bit `XxHash64` collision between *different* sequences is a separate matter. At ~ 10^4 blocks the birthday probability is ~ 10^{-11}, and the consequence is merely a different-but-still-deterministic ordering, not incorrect output, since the identifier recovered from the name is exact. No mitigation needed.

1.6.2 Consequences of the identifier lookup table

Replacing `BlockId::from_str(&paf.q.name)` with a table lookup has a wider blast radius than it first appears:

- **Signature churn.** `Alignment::from_minimap_paf_obj` (`align/minimap2_lib/align_with_minimap2_lib.rs:89`) and `Alignment::from_paf_str` (`align/mmseqs/paf.rs:40`) must both take `&BlockNames`. `find_matches` and `filter_matches` (`pangraph/graph_merging.rs:176`, `:187`) gain the parameter, and `self_merge` constructs the table once per iteration from `graph.blocks`.
- **The mmseqs path breaks silently otherwise.** `PafTsvRecord` deserialises `query: BlockId` and `target: BlockId` directly through serde (`align/mmseqs/paf.rs:15`, `:20`), which only works while names are bare decimal integers. With canonical names this must become a `String` field parsed through `id_of`. Missing this would turn a working backend into a parse error — caught by compilation only if the field type is changed deliberately.
- **Lookups become fallible.** An unknown name currently cannot happen; with a table it can, so the error path needs a real internal error rather than an `unwrap`. This is a strict improvement in diagnosability.
- **Cost is negligible.** The table is `O(n_blocks)` entries (hundreds to low thousands), rebuilt once per `self_merge` iteration. Hashing all consensuses costs one pass over the block sequence set — a few Mbp at `XxHash64` throughput, i.e. milliseconds against multi-second alignment stages.

1.6.3 The mmseqs backend did not carry the defect

An earlier draft of this note claimed that `align_with_mmseqs` carried the identical defect by a different route, since it also writes its FASTA in `BTreeMap` order under `id.to_string()` names. Measurement contradicts that. On the same three **Campylobacter** genomes that swing minimap2 from 508 to 462 blocks, the mmseqs backend was already order-invariant before any change:

backend / argument order	blocks (before fix)	consensus-set digest
mmseqs, `ERS AP NZ`	297	`865e8320ea1d72b1`
mmseqs, `NZ ERS AP`	297	`865e8320ea1d72b1`

The reason is precisely the `-X` flag. mmseqs does **not** pass it, so it computes both directions of every pair and the resulting alignment set is symmetric - independent of which sequence is nominally the query. `filter_matches` then ranks by energy, and the ranking is itself order-independent, so no tie-break is ever reached. `MM_F_NO_DUAL` is not merely one mechanism among several: it is **the** mechanism, and only the minimap2 path used it.

The canonicalization is still applied to mmseqs, for two reasons: it removes the backend's residual reliance on vector-order tie-breaking in `filter_matches` (latent rather than observed), and it keeps the two backends behaving identically rather than accidentally-equivalently. But it should be recorded as hardening, not as a bug fix.

It is not free, though. Because the anchor tie-break changed, mmseqs output **moves**: 297 blocks before, 317 after, order-invariant in both cases. That is a different-but-equally-valid anchor choice on a backend that was not broken, and anyone comparing mmseqs graphs across this change should expect the shift.

This diagnosis has a direct consequence: if dual mapping is what protected mmseqs, then enabling it for minimap2 should protect minimap2 too, without any canonical naming. That prediction was tested, and it holds — see the dual-mapping alternative section.

1.6.4 Downstream fixtures and `pypangraph`

`pypangraph` needs no code change. Its schema (`pypangraph/pangraph_schema.py`) is autogenerated from the Rust types and covers only the serialised graph — `Pangraph`, `PangraphPath`, `PangraphBlock`, `PangraphNode`, `Edit` — with no reference to `Alignment`, `Hit` or the PAF types. `BlockId` remains a `uint`, so the loader, `IndexedCollection` and the integer/string identifier duality are all untouched.

What does need attention is *fixture regeneration*. `packages/pypangraph/tests/data/plasmids.json` is committed data, so it does not move when the build changes — but the moment it is regenerated with a fixed binary, block boundaries, block counts and identifiers all shift, and the hard-coded expectations in `tests/test_graph.py` break:

location	pinned expectation
`test_graph.py:101`	literal block id `"14710008249239879492"`
`test_graph.py:61-63`	137 blocks, 27 core, 10 duplicated
`test_graph.py:88-90`	137 × 15 matrix, sum 1042

The same caveat applies to `data/test_graph.json` on the Rust side if it is ever rebuilt rather than merely consumed. Neither fixture is regenerated by this change, so nothing breaks on landing; the point is that regeneration must be a deliberate, separate step, with the assertions above updated in the same commit rather than rediscovered as a mysterious test failure later.

Worth recording in `pypangraph`'s changelog regardless: even after the structural guarantee lands, `BlockId` values still derive from `fasta.index`, so downstream code must not assume identifiers are stable across runs with different input order.

1.6.5 Things that are safe

- **Correctness of merging is unaffected.** Which block is anchor changes which valid consensus is retained, not whether the result is valid. Sequence reconstruction and the existing sanity checks are indifferent to the choice.
- **Integration tests do not exercise this path.** `data/test_graph.json` is consumed by the export tests (`itest_export_*.rs`), which read a pre-built graph. They will not shift.
- **No serialised format changes.** `Hit.name` remains a `BlockId`; canonical names exist only for the duration of an alignment call and never reach disk.
- **No performance regression.** `-X` still halves the pair count; the index size and mapping work are unchanged; the parallel stage gets marginally better splitting.

1.6.6 Neighbor-joining ties are not rare, and had to be fixed too

An earlier draft listed the neighbor-joining tie-break as an out-of-scope residual, on the assumption that exact ties in the Q matrix are rare with real-valued mash distances. That reasoning was wrong, and for the commonest small case it is wrong by construction.

For three taxa, write S_k for the sum of row k of the distance matrix. With $n = 3$ the code computes $Q_{ij} = D_{ij} - S_i - S_j$, so

$$Q_{12} = D_{12} - (D_{12} + D_{13}) - (D_{12} + D_{23}) = -(D_{12} + D_{13} + D_{23})$$

and the same value comes out for Q_{13} and Q_{23}. *Every* off-diagonal entry is identical, regardless of the data. `Q.argmin()` therefore always returns `(0, 1)`, and the tree is always `((first, second), third)` in argument order. This is exactly what the reproducer shows: the same three genomes give `((ERS990151,AP025961),NZ_CP035927)` in one order and `((NZ_CP035927,ERS990151),AP025961)` in another - genuinely different topologies, not sibling swaps.

Since a different topology legitimately yields a different graph, canonicalizing the aligner boundary cannot help here; the tree itself has to stop depending on argument order. The fix is to order the leaves handed to neighbor joining by a content-derived key, so that the row order the tie-break falls back on is a property of the sequences rather than of the command line:

```
fn graph_content_key(graph: &Pangraph) -> Vec<u64> {
  graph.blocks.values().map(consensus_hash).sorted().collect()
}

pub fn build_tree_using_neighbor_joining(graphs: Vec<Pangraph>) -> Result<...> {
  let mut graphs = graphs;
  graphs.sort_by_cached_key(graph_content_key);
  // ...
}
```

Without this, the default invocation - no `--guide-tree` - remains order-dependent, so a fix that stopped at the aligner boundary would have left the common case broken.

1.6.7 Residual order dependence after the fix

Honesty about what is *not* fixed:

1. Identical-consensus blocks, as discussed, retain identifier-level (not sequence-level) order dependence.
2. `BlockId` values themselves still derive from `fasta.index`, so the identifiers appearing in output JSON differ between argument orders even when the graph structure is identical. If byte-identical output is wanted, singleton identifiers would also need a content-derived assignment — worth considering, but orthogonal to the correctness issue.

With those caveats, the guarantee becomes: *the graph structure is a pure function of the input sequence set, the tree topology, and the alignment parameters* — independent of argument order and of sibling order within the tree.

1.7 Validation results

Implemented on branch `debug/order-dependence` across five commits: `BlockNames` and its unit tests, the aligner boundary, the downstream tie-breaks, the neighbor-joining leaf ordering, and the invariance regression tests.

1.7.1 The reproducer collapses

All six permutations of the three **Campylobacter** genomes, `--circular --guide-tree`:

argument order	blocks before	blocks after
`ERS AP NZ`	508	512
`ERS NZ AP`	462	512
`AP ERS NZ`	497	512
`AP NZ ERS`	452	512

argument order	blocks before	blocks after
`NZ ERS AP`	462	512
`NZ AP ERS`	452	512

The consensus multisets are byte-identical across all six (SHA-256 `5495a5963cd3b2b8`), as are the depth multisets. A sibling-swapped guide tree gives the same digest, and the neighbor-joining path (no `--guide-tree`) likewise collapses to a single answer across all six orders - 522 blocks, which differs from 512 only because it infers a different topology, as it should.

The JSON is **not** byte-identical between orders, exactly as predicted: `BlockId` still derives from `fasta.index`, so identifiers move even when structure does not.

1.7.2 The regression tests discriminate

`packages/pangraph/tests/itest_build_order_invariance.rs` asserts invariance over six argument permutations under three topologies, plus sibling-swap invariance. Each assertion was checked against deliberately reverted code:

reverted component	failing case
canonical names + anchor tie-break	`balanced_guide_tree`, order `[3,2,1,0]`
neighbor-joining leaf ordering	`no_guide_tree`, order `[3,2,1,0]`

One lesson is worth recording, because the first version of this test was worthless. A *three*-genome fixture cannot detect the anchor defect: the final merge joins a depth-2 clade with a depth-1 leaf, so depth decides and the tie-break never runs. The test passed with the fix fully reverted. It needs four genomes and a balanced topology, so that the top merge is depth-2 against depth-2 - a genuine tie. Any future fixture for this class of bug must be checked against reverted code before it is trusted.

1.7.3 Fixtures are untouched

`data/test_graph.json` and `packages/pypangraph/tests/data/plasmids.json` are byte-identical after the change, so both suites verify the fix rather than absorb it. All 327 unit tests and all integration tests pass; `clippy --all-targets -Dwarnings` is clean.

1.7.4 Still unverified

The two-identical-consensus-blocks merge case is covered only at the naming level (`identical_consensus_blocks_get_distinct_names`), not end to end through a real alignment. The collision hazard argument in the corresponding section is therefore reasoned, not measured.

1.8 The dual-mapping alternative

The finding that mmseqs was never affected raises an obvious question: if computing both directions is what made mmseqs immune, could minimap2 simply be told to do the same — fixing the defect without any of the naming machinery? The answer is yes, with caveats that decide whether it is the right trade.

1.8.1 Disabling `-X` outright does not work

`X` is a bundle, not a toggle. In the in-tree wrapper (`packages/minimap2/src/options_args.rs:324`) a single boolean sets four flags:

```
  if args.X {
    map_opt.flag |= (MM_F_ALL_CHAINS | MM_F_NO_DIAG | MM_F_NO_DUAL | MM_F_NO_LJOIN) as
i64;
  }
```

and the `asm5`/`asm10`/`asm20` presets pangraph uses set none of them (unlike the `ava-*` presets, `packages/minimap2-sys/minimap2/options.c:96`). So `X: false` also discards all-chains and no-diagonal reporting. Measured on the three **Campylobacter** genomes, the result is order-invariant and useless:

variant	blocks	invariant across six orders
`X: false`	3	yes
`-X` with only `MM_F_NO_DUAL` cleared	459	yes

Three blocks for three genomes means nothing merged at all. The likely mechanism — inferred from minimap2′s secondary-alignment filtering rather than instrumented — is that without `NO_DIAG` each sequence's perfect self-hit becomes the primary chain, and cross-genome hits are then discarded as low-ratio secondaries.

1.8.2 Enabling dual mappings does work

The precise variant is minimap2′s `-X --dual=yes`: keep all-chains, no-diagonal and no-long-join, clear only `MM_F_NO_DUAL`. Tested against the *entire pre-fix codebase* — `BlockId` sequence names, `BlockId` ordering, reference-wins-ties anchor selection, input-order neighbor-joining leaves — all six argument permutations produced 459 blocks with an identical consensus-set digest (`e33506c21d945b73`). No canonical naming, no lookup table, no tie-break changes.

This is a genuinely simpler fix. It needs one wrapper change: split `X: bool` into `X` plus a separate `dual` option, since nothing currently exposes `--dual`.

1.8.3 A confounded comparison, corrected

An earlier version of this section reported that dual mappings gave 459 blocks against 512 for the landed fix, and reasoned from that gap that dual mappings produce a better-consolidated graph. That comparison was *confounded*: it set pre-fix code + dual mappings against canonical naming + `-X`, changing two things at once. The 459 was a property of the pre-fix anchor tie-break, not of dual mappings.

Holding canonical naming fixed and toggling only dual mappings, the same three genomes give 512 blocks against 514 — a difference of two blocks, not fifty-three. The quality argument built on the larger gap does not survive.

1.8.4 Benchmark across the bundled datasets

Both variants were run over every dataset in `data/`, plus the three-genome reproducer, with canonical naming held fixed so that dual mapping is the only variable. Graph metrics are computed from the JSON directly; the definitions were cross-checked against `pypangraph.to_blockstats_df()` on `data/test_graph.json` and agree exactly (14 blocks, 6 core, 33 611 core bp, 50 791 pangenome bp). A block counts as *core* when every path crosses it exactly once.

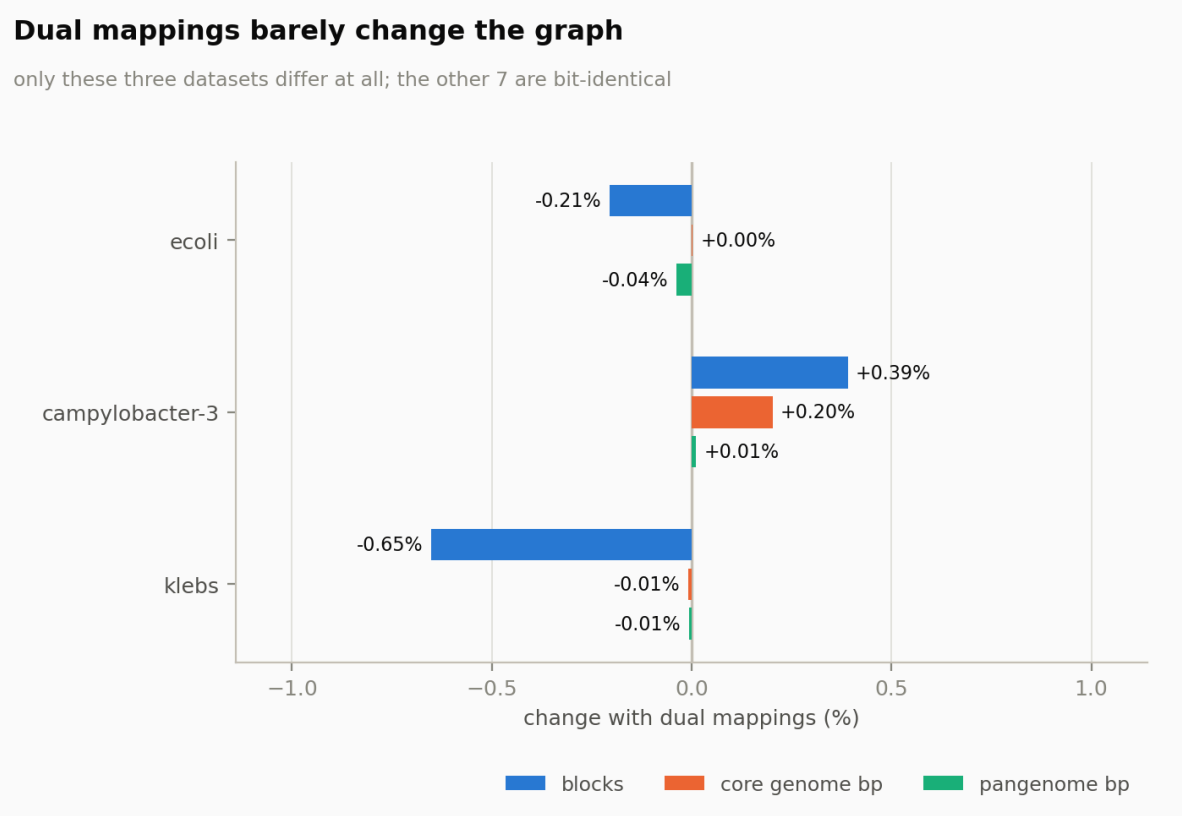

Figure 1: Graph metrics are unchanged on seven of ten datasets — `example`, `flu-h1`, `flu-h3`, `ges-1`, `mpox`, `russian_doll_plasmids` and `sc2` are bit-identical. Where they do move, every change is under 1%, and not consistently signed.

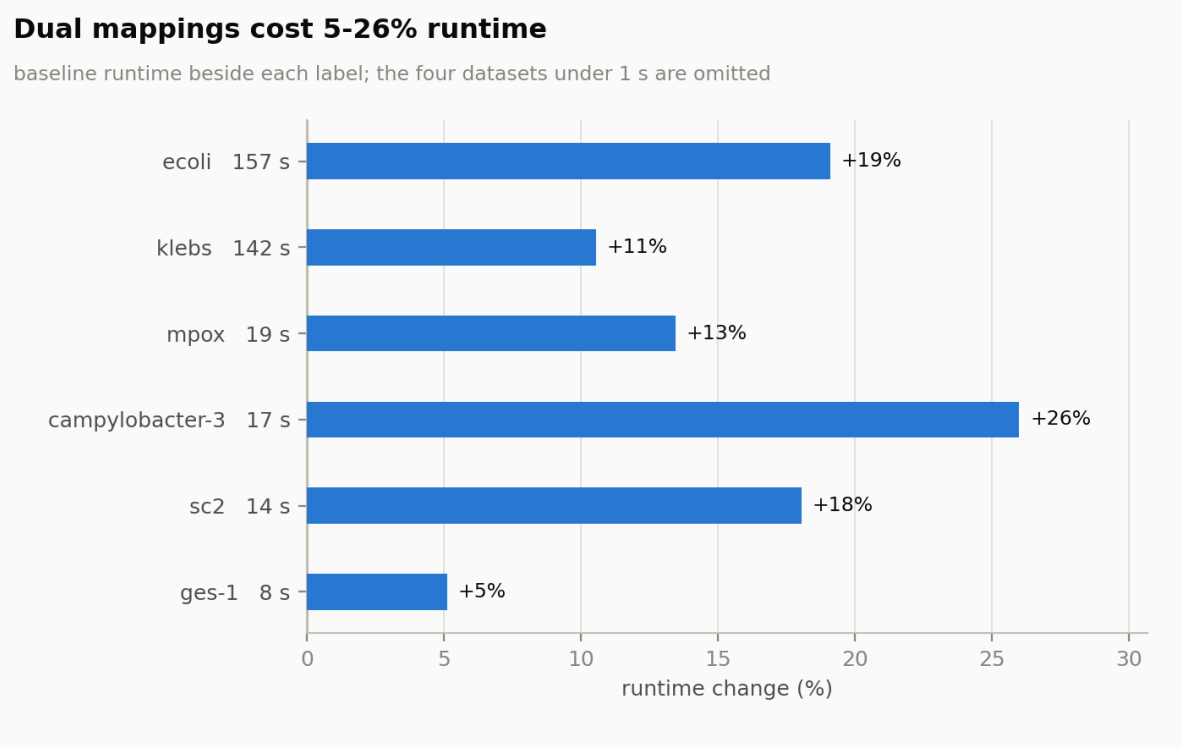

Figure 2: Runtime overhead of dual mappings. Datasets completing in under a second are omitted as timing noise. Measured with nothing else running on the machine.

Absolute values for the datasets where anything changed:

dataset	variant	blocks	core bp	pangenome bp
klebs (9 genomes)	`-X`	1 381	4 458 249	7 644 985
	dual	1 372	4 457 866	7 644 453
ecoli (10 genomes)	`-X`	2 914	3 782 006	7 830 290
	dual	2 908	3 782 120	7 827 250
campylobacter (3)	`-X`	512	102 659	3 445 626
	dual	514	102 867	3 445 993

Two things follow. Dual mappings are close to free in graph terms — seven of ten datasets are bit-identical, and the three that move do so by well under 1%. And the direction of the change is inconsistent: dual mappings *reduce* the block count on klebs (− 0.65%) and ecoli (− 0.21%) but *increase* it on Campylobacter (+ 0.39%). There is no evidence here that either variant consolidates the graph better; they simply make marginally different choices at a handful of boundaries.

The cost, by contrast, is real and consistent: every dataset gets slower, by 5% (`ges-1`) to 26% (Campylobacter), with the two large bacterial pangenomes at + 11% and + 19%. That is well short of the 2× one might assume — alignment is not the whole pipeline — but it is a uniform tax for no measured graph benefit.

The remaining argument for dual mappings is therefore the one from evidence rather than from outcome: the direction probe showed the two directions genuinely find different homology (147 versus 146 hits, 263 139 versus 266 047 matched bases), and under `-X` one of them is discarded arbitrarily. Using both is more principled. It just does not, on these datasets, produce a measurably different graph.

1.8.5 What it does not buy

The decisive distinction is elsewhere. Dual mappings do not remove the order-sensitive tie-breaks; they merely stop them being reached. The energy sort still falls through to vector order on exact ties, and reference-wins-ties would still privilege one side. Mirror alignments happen to score slightly differently, so no tie arises in practice — which is exactly why mmseqs looked immune. *Dual mappings make invariance a property of the data; canonical naming makes it a property of the code.* If two mirror alignments ever scored equal, the order dependence would return.

1.8.6 Recommendation

Keep the canonical naming, and do not adopt dual mappings on the strength of this evidence.

The case for dual mappings rested on two claims, and the benchmark removes both. It was to be the simpler fix — but it is not sufficient on its own, since the neighbor-joining ordering still has to be fixed separately, and it only makes invariance contingent on mirror alignments never tying. And it was to produce a better graph — but across ten datasets it produces the *same* graph seven times, sub-1% differences three times, in no consistent direction, at a uniform 5–26% runtime cost.

What remains is a principled objection to `-X`: discarding one direction of a pairwise alignment throws away real signal. That objection stands, and if the alignment stage is ever revisited for quality reasons, dual mappings are the right thing to reach for — ideally with a quality metric better than block counts to judge by. On present evidence it is a cost with no measured benefit, and the canonical naming already delivers the guarantee it was meant to provide.